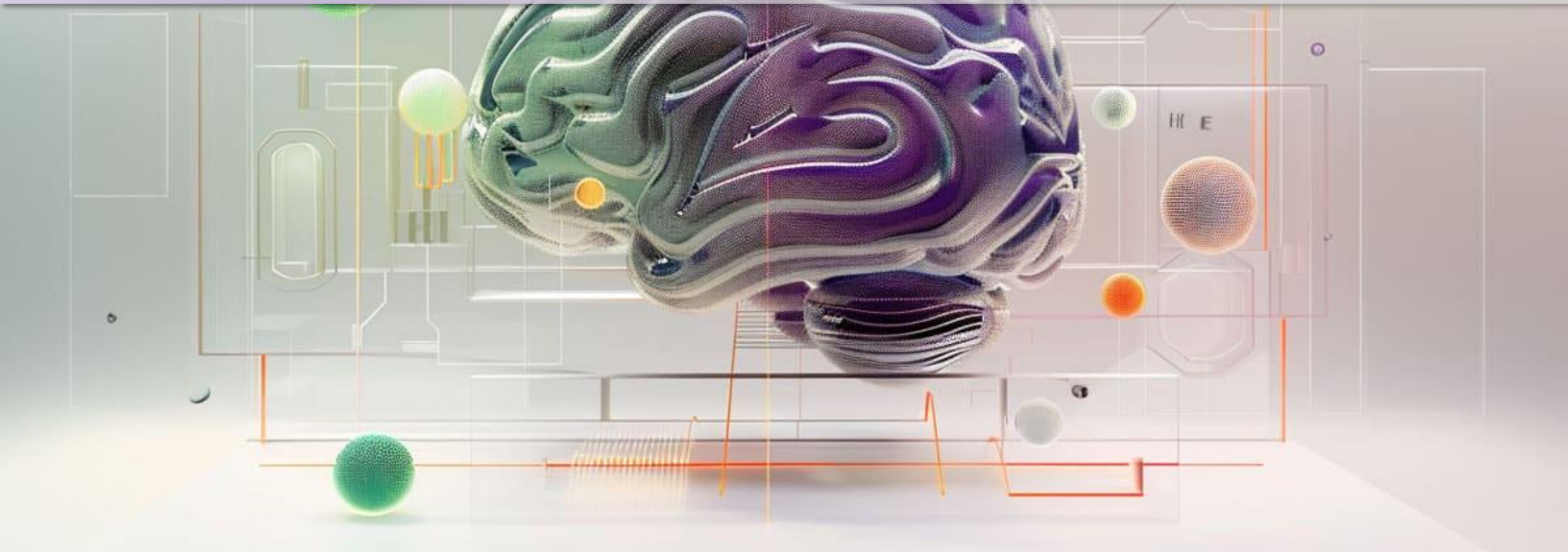


HFA - HBM Fast Allocator

NW4AI team, Pnueli Lab, TRC



Executive Summary – HBM Fast Allocator (HFA)

- We developed a **high-performance** HBM memory allocator (**HFA**) for Ascend
- Consistent ~99.7% HBM utilization across all object sizes
 - **ACL-Native** peaks ~99.5%, but drops to 3-25% on small allocation
 - **Torch-NPU allocator (NPUCachingAllocator)** reaches only ~70% utilization on average
- HFA has ultra-low, stable **allocation/free latency (~0.17 μ s)**
 - ACL-Native spikes to 170,000-418,000 μ s (>300,000x slower), causing severe performance stalls.
 - Torch-NPU allocator is ~3X slower than HFA
- Enables large models, fewer OOM events, and smoother LLM execution
- **Proposal:** Integrate HFA into HQ UMDK next-gen AI stack, and push it to open source (UMDK and torch-npu).

Use Case – Why a New HBM Allocator Matters

- **Inference Today is Highly Dynamic:**
- **AI models create unpredictable memory demands.**
- **Key Temporary Memory Consumers:**
 - Infer-Shape Memory: frequent, short-lived, high churn
 - Intermediate Results: vary by input; require heavy reuse.
 - Workspace Buffers: large and reallocated often.
- **Why Standard Allocators Struggle:**
 - Fragmentation due to dynamic sizes.
 - Workspace peaks can cause OOM.
 - Parallel execution increases contention.
- **Business Impact:**
 - Lower throughput: wasted HBM; OOM risk.
 - With improved allocator we get:
 - Faster inference (high thruput / low latency)
 - More usable memory (reduce fragmentation), less \$\$\$ spent on HBMs
 - Support larger models & larger batch size
 - (Because less time is spent on alloc/free + less fragmentation)

HFA design goals

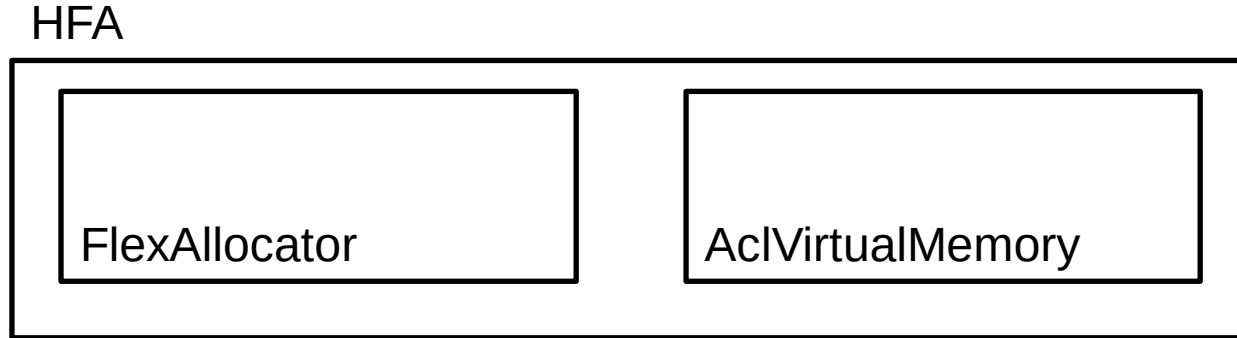
- **Main goal is to reduce HBM memory utilization.**
 - Therefore SoTA designs were rejected:
 - **Buddy Allocator:** since space is wasted when allocation is served by a block of size nearest power of 2.
 - Average case example (uniform size distribution): **75% utilization**
 - Worst case example: **50% utilization**
 - **Slab Allocator:** **75%-95% utilization** (depending on tuning and use case).

HFA memory utilization is > 99% in all cases

- **Secondary goal is to maintain very low alloc/free latency.**

Architecture

HFA: structure

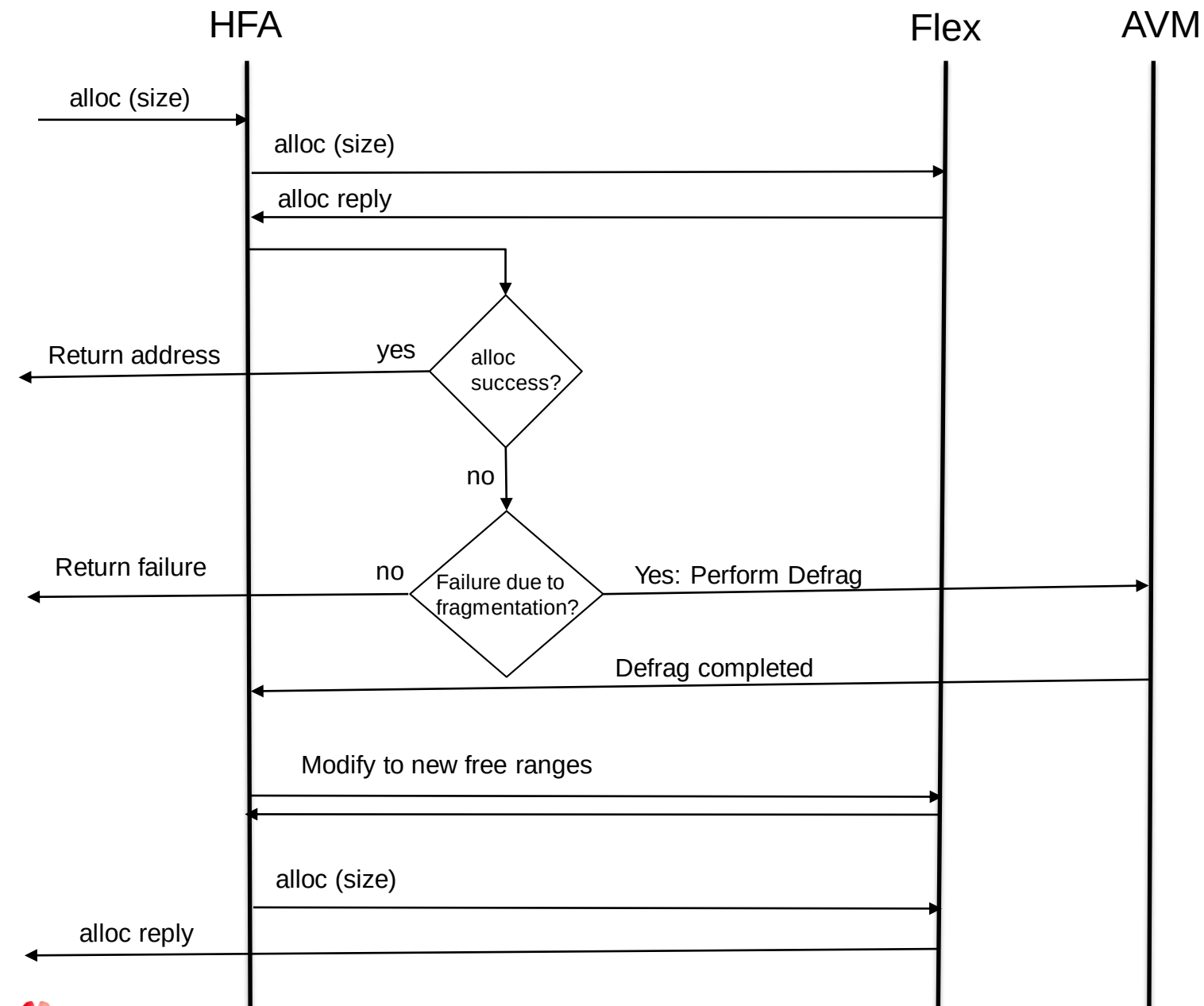


HFA uses FlexAlloc and AclVirtualMemory.

- AclVirtualMemory:
 - Allocates virtual address space and maps HBM physical pages into it.
 - Resolves fragmentation in virtual address space.
- FlexAlloc:
 - Takes N contiguous HBM virtual address ranges and splits them into smaller objects, on-the-fly, according to user's allocation requests.
 - Merges adjacent free objects into single contiguous object

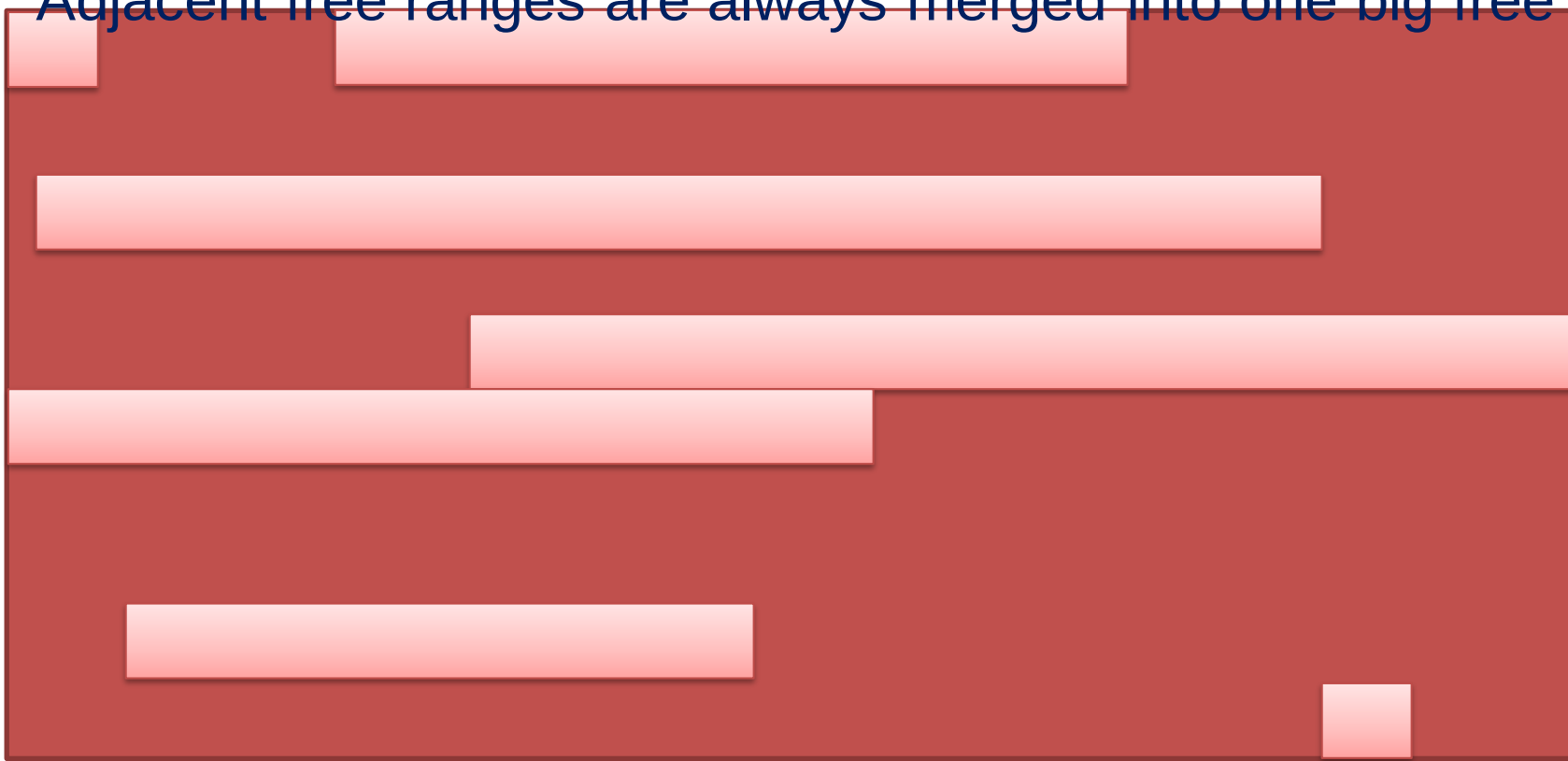
```
class HbmFastAlloc {  
    AclVirtualMemory *mp_avm { nullptr };    ///< Pointer to the AclVirtualMemory object  
    FlexAlloc *mp_fa { nullptr };            ///< Pointer to the FlexAlloc object  
    std::unordered_map<uint8_t *, std::pair<size_t, size_t>> m_alloc_map;    ///< Map of allocated memory  
};
```

HFA: allocation workflow



Flex: Region memory layout

- No division into blocks, slabs, buddy-pairs
- An object of any size can be located in any offset in HBM's virtual address range.
- Adjacent free ranges are always merged into one big free range.



Flex: Data Structures

For each HBM region we use:

1. StartMap: An ordered map of all free ranges, ordered by start offset. StartMap is consulted upon object free, to meld the adjacent ranges into one.
2. SizeMap: An ordered multimap of all free ranges grouped by size. SizeMap is consulted upon object alloc to find the smallest range that is $\geq$ requested_size.
3. SizeMap's value is a valid iterator inside StartMap. For alloc/free efficiency.

```
struct Range {  
    length_t size;    // length of this free range  
};  
  
// All free ranges, ordered by start offset.  
using StartMap = std::map<offset_t, Range>;  
  
// Index of free ranges, ordered by size.  
// Value is an iterator into StartMap (the main storage).  
using SizeMap = std::multimap<length_t, typename StartMap::iterator>;  
  
StartMap m_free_ranges_by_offset;  
SizeMap m_free_ranges_by_size;
```

Flex: Allocation Logic

Given requested allocation size (R) is rounded up to nearest 64B modulo size (S). Then, S is used as lookup key in `m_free_ranges_by_size` multimap:

- if match found (`key==requested_size`), remove the entry from `m_free_ranges_by_size` multimap and return offset and size to caller.
- If match not found:
 - If any entry was found, its key (size) is bigger than `requested_size`, so remove the entry from `m_free_ranges_by_size` multimap and return offset and size to caller.
 - If no entry was found, return allocation failure.

(*) If entry was found in `m_free_ranges_by_size` multimap, the corresponding entry is also removed from `m_free_ranges_by_offset` map.

AVM (ACL Virtual Memory) - Logic

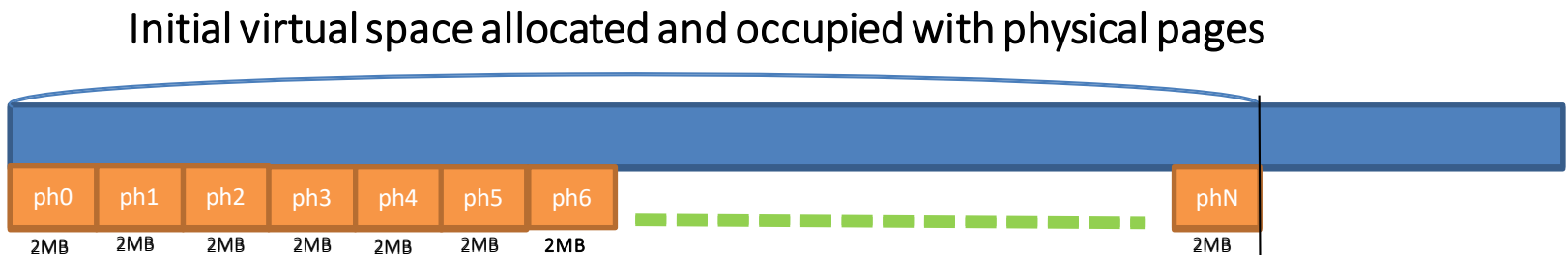
- The **AVM allocates** at initialization stage, the maximum virtual & physical memory available in the NPU.
- All available **physical pages are mapped** to the beginning of the virtual memory space.
- **Flex uses the virtual address range** allocated by AVM as the memory resource for providing memory objects to the user.
- Memory objects are allocated continuously on the virtual memory space, with no additional `aclrtXXX` calls, thus introducing minimum latency for alloc/free operations.
- AVM **handles *defrag*** as required

AVM (ACL Virtual Memory) - Logic

Example of defrag process

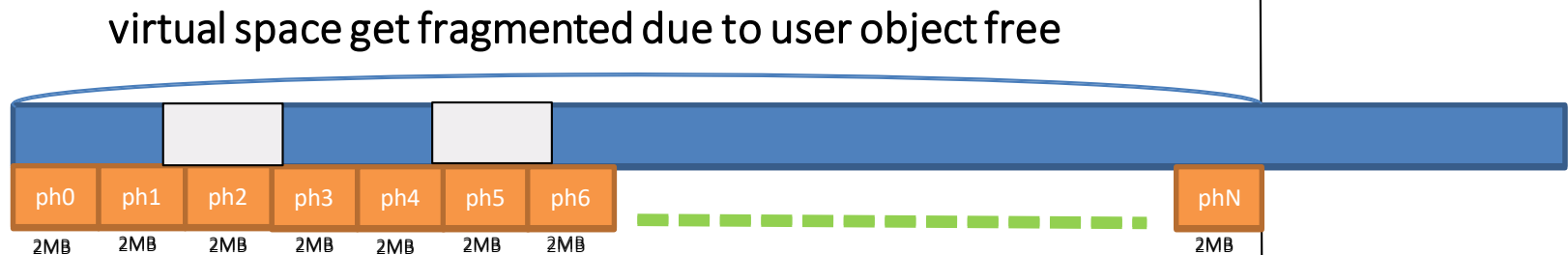
1. Initial allocation

Virtual memory address space
Physical memory pages



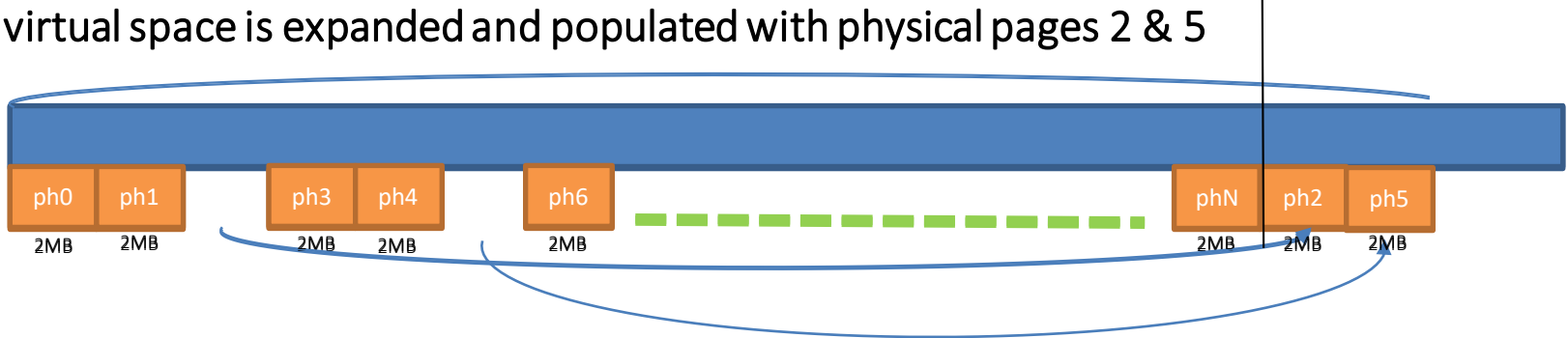
2. User creates fragmentation

Virtual memory address space
Physical memory pages



3. Defragmentation

Virtual memory address space
Physical memory pages

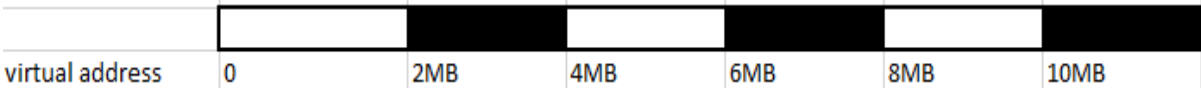


HFA Defrag (example)

- HFA defrag converts small fragments into a new big contiguous range in virtual address space.

Free ranges	
Allocated ranges	
don't use ranges	

Before defrag: 3 distinct small ranges, 2MB each



After defrag (one big contiguous free range of 6MB)

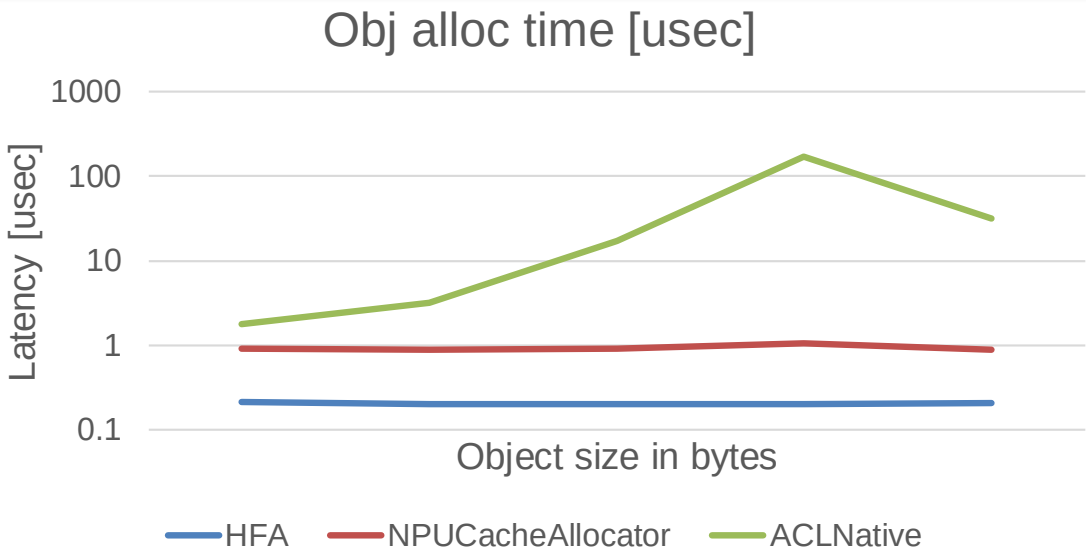


Performance Benchmark – HFA – static sizes

HFA vs Torch-npu vs Acl native

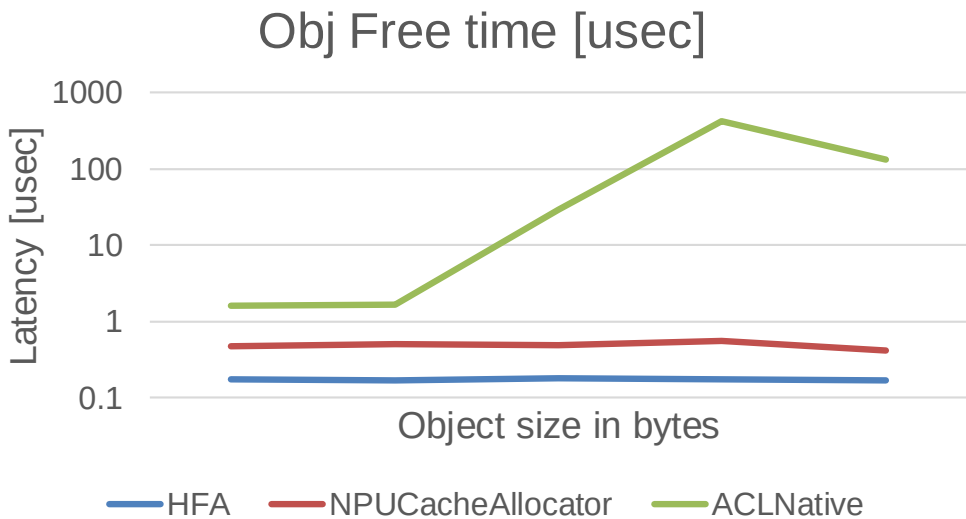
Object Allocation Performance comparison

Object Size bytes	HFA Average allocation time (usec)	NPUCachingAllocator Average allocation time (usec)	ACLNative Average allocation time (usec)
1000	0.212	0.909	1.791
10000	0.201	0.881	3.219
100000	0.201	0.916	17.152
1000000	0.202	1.065	170.006
10000000	0.210	0.885	31.831



Object Free Performance comparison

Object Size bytes	HFA (usec)	NPUCachingAllocator (usec)	ACLNative (usec)
1000	0.175	0.481	1.604
10000	0.172	0.498	1.659
100000	0.178	0.497	28.757
1000000	0.174	0.566	418.427
10000000	0.172	0.414	132.831



Ascend 910B4

Performance Benchmark – HFA – vLLM torch_npu inference

HFA vs NPUCachingAllocator

Average alloc/free latency

Ascend 910B4

Action	NPUCachingAllocator [usec]	HFA [usec]	% improvement
Alloc	3.03	0.86	71.6%
Free	1.315	0.720	45%

Total alloc/free latency and memory utilization

Test	Avg. prompts memory latency(sec) NCA / HFA	avg. Prompts utilization(%) NCA / HFA	Avg. TTFT(sec) NCA / HFA	Avg. e2e(sec) NCA / HFA
synchronous test: HFA64 prompts = 10 Input = 16384 tokens output = 9182 tokens	14.447 / 6.879	99.98 / 99.999	0.352 / 0.334	326.1 / 308.4
random dataset: HFA1024 prompts = 10 input = 16384 tokens output = 1024 tokens	4.19 / 1.69	99.96 / 99.999	8.231 / 7.947	53.75 / 48.72
sonnet dataset: HFA1024 prompts = 10 input = 16384 tokens output = 1024 tokens"	4.10 / 1.74	99.96 / 99.999	13.194 / 12.656	64.47 / 63.21

(**) NCA utilization is good mainly due to Paged-Attention which uses fixed-size blocks

Full Results: <https://onebox.huawei.com/v/1b16c4f9bd164e8861d89cc1030bd7b1?type=0>

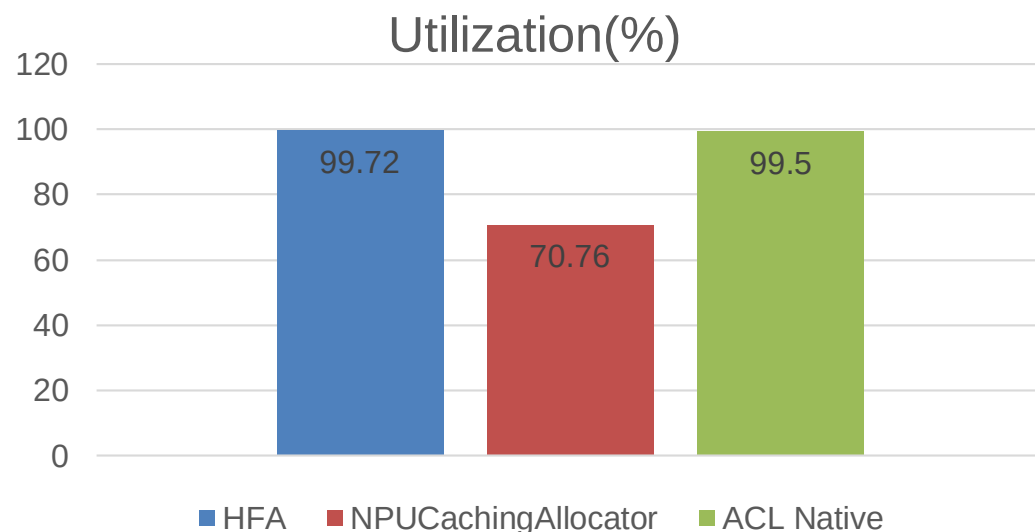
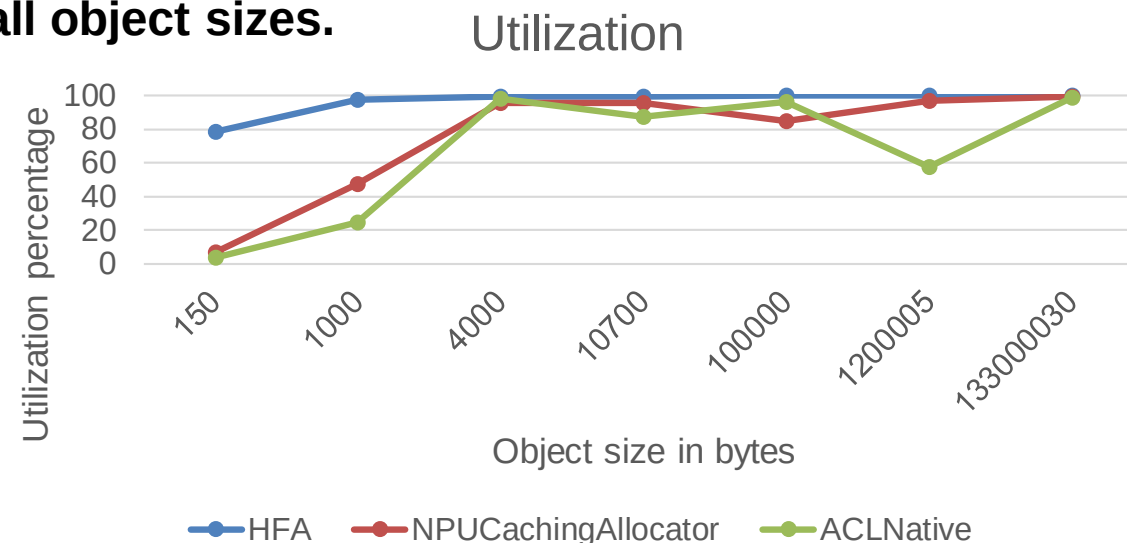
Performance Benchmark – Memory Utilization

HFA vs Torch-NPU vs Acl native

Utilization comparison

Object Size bytes	HFA utilization (%)	NPUCachingAllocator Utilization (%)	ACLNative Utilization (%)
150	78.12%	7.15%	3.6%
1000	97.6%	47.68%	24.5%
4000	99.2%	95.37%	98.2%
10,700	99.52%	95.37%	87.2%
100,000	99.97%	85.04%	95.9%
1,200,005	99.99%	96.98%	57.3%
133,000,030	99.95%	99.09%	98.8%

HFA Achieves ~99.7 memory utilization, Highest across all object sizes.



Scenario:

1. allocated 27 objects (ranged from 1MB-1GB), allocated total of 10737332006 bytes (~10GB)
2. freed every second object - 4440204957 bytes (freed 14 objects)
3. allocated 1 "big" object of size 4515168256

Performance Benchmark – Summary of Utilization & Latency

Allocator	Utilization	Latency
ACL Native	Mid (good, use case dependent)	Very High (very bad)
NPU Caching Allocator	Mid (ok, use case dependent)	Low (good)
HFA	High (very good)	Very Low (very good)

Future work

Code :

<https://gitlab-il.rnd.huawei.com/tubes/hbm-fast-alloc>

Future work:

1. Add policy for physical pages allocation:
 - a) Allocate entire HBM at init phase.
 - b) Expand: Allocate part of HBM at init, then grow or shrink on demand
2. Add cache for small sizes and remote allocations
3. Push to open source (UMDK, torch-npu).
4. Find project that can benefit from HFA (inference, training, other.).

Appendix A: FlexAlloc - allocation example

Flex example: Data structures (initial) (*)

```
m_free_ranges_by_size = {{1024, 0}}
```

This means we have a single free range of size 1024 bytes that starts at offset 0

size	offsets
1024	0

(*) Note: in this simple example we use:

1. Region size: 1024B
2. Object size is in multiples of 64B

Both are configurable

Legend	
Free ranges	
Allocated ranges	

0	64	128	192	256	320	384	448	512	576	640	704	768	832	896	960	1024
---	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	------

Flex example: Data structures (cont.)

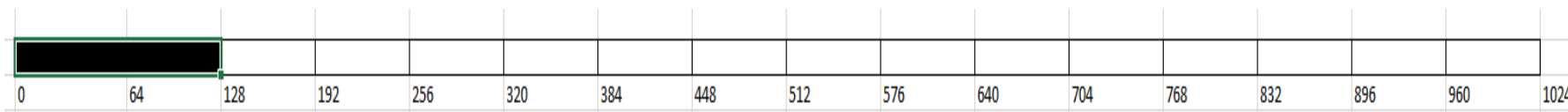
After allocating object of size 128B

```
m_free_ranges_by_size = {{896, 128}}
```

This means we now have a single free range of size 896B that starts at offset 128

size	offsets
896	128

Legend	
Free ranges	
Allocated ranges	



Flex example: Data structures (cont.)

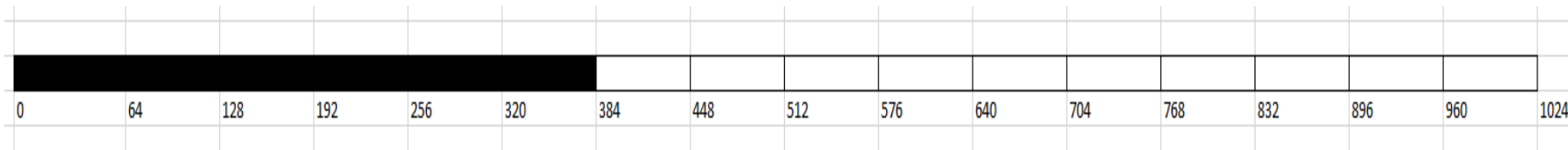
After allocating 4 objects of size 64B

```
m_free_ranges_by_size = {{640, 384}}
```

This means we now have a single free range of size 640B that starts at offset 384

size	offsets
640	384

Legend	
Free ranges	
Allocated ranges	



Flex example: Data structures (cont.)

After free object of size 64B at offset 192

```
m_free_ranges_by_size = {{64, 192}, {640, 384}}
```

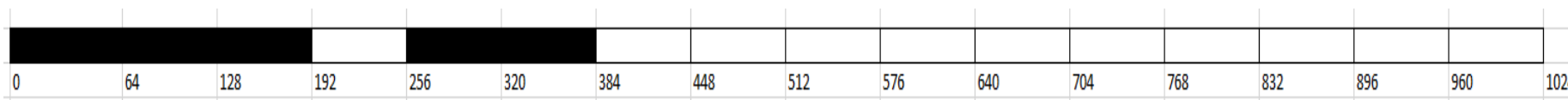
This means we now have 2 free ranges:

size 64B that starts at offset 192

size 640B that starts at offset 384

size	offsets
64	192
640	384

Legend	
Free ranges	
Allocated ranges	



Flex example: Data structures (cont.)

After free object of size 64B at offset 256

```
m_free_ranges_by_size = {{128, 192}, {640, 384}}
```

This means we now have 2 free ranges:

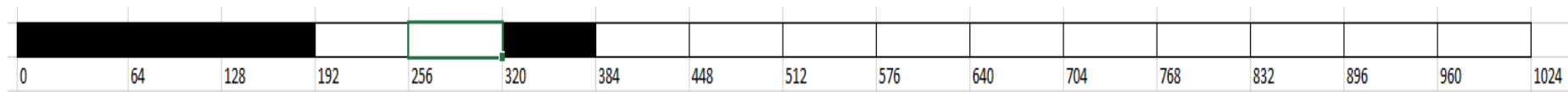
size 128B that starts at offset 192

size 640B that starts at offset 384

Note: 2 free ranges of size 64B were melded to create one free range of size 128B

size	offsets
128	192
640	384

Legend	
Free ranges	
Allocated ranges	



Flex example: Data structures (cont.)

After free object of size 128B at offset 0

```
m_free_ranges_by_size = {{128, 0}, {128, 192}, {640, 384}}
```

This means we now have 3 free ranges:

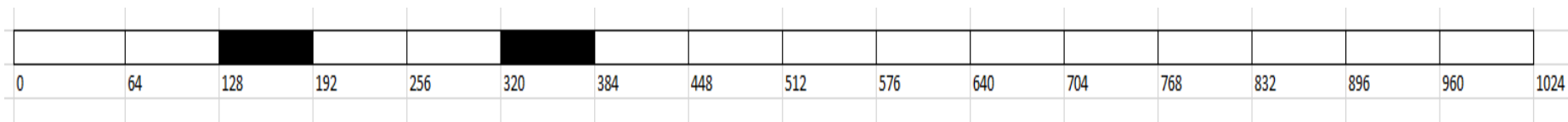
size 128B that starts at offset 0

size 128B that starts at offset 192

size 640B that starts at offset 384

Size	offsets
128	192, 0
640	384

Legend	
Free ranges	
Allocated ranges	



Flex example: Data structures (cont.)

After free object of size 64B at offset 128

`m_free_ranges_by_size = {{320, 0}, {640, 384}}`

This means we now have 2 free ranges:

size 320B that starts at offset 0

size 640B that starts at offset 384

Note: 3 temporary free ranges `{{128, 0}, {64, 128}, {128, 192}}` were melded into one `{{320, 0}}`

Size	offsets
320	0
640	384

Legend	
Free ranges	
Allocated ranges	

0	64	128	192	256	320	384	448	512	576	640	704	768	832	896	960	1024